

Module A — API — "ReClaim"

Duration: 3 hours 3 小時

Stack: Any server-side framework + MySQL

任意伺服器端框架 + MySQL

Scenario 情境

Shanghai Pudong International Airport (PVG) has commissioned **ReClaim**, a single lost property service for the whole airport. Lost property agents register found items and process passenger claims from any terminal; passengers report lost items online and follow their claim's progress; the system suggests which stored items best match each new claim.

上海浦東國際機場 (PVG) 委託開發「ReClaim」全機場統一失物招領服務系統。失物處理人員可在任意航廈登記拾獲物品並處理旅客的申報；旅客則能透過線上系統申報遺失物並追蹤進度；系統亦會針對每一筆新申報，自動推薦最符合的物品。

The staff desk application and the passenger portal are built by other teams. You deliver the REST API behind them, implemented exactly as specified below. Your implementation will be tested against an automated test suite (Bruno).

Responses must match the expected structure, status codes, and business logic precisely.

地勤櫃檯應用程式與旅客端入口網站由其他團隊負責開發。你的任務是交付其背後運行的 REST API，並嚴格按照以下規格實作。系統會使用自動化測試套件 (Bruno) 來驗證你的作品。API 回應必須完全符合預期的結構、狀態碼與業務邏輯。

Setup 設定

- Import the provided database dump: dist/database/reclaim-db.sql (MySQL). The same dataset is also provided as JSON files in dist/data/json/ as a failsafe — the SQL dump is the primary source and the assessment dataset is the MySQL database
請匯入提供的資料庫 Dump 檔：dist/database/reclaim-db.sql (MySQL)。dist/data/json/ 目錄下亦提供相同的 JSON 資料集作為備援方案 — 請注意 SQL Dump 為主要資料來源，評分用的資料集以

MySQL 資料庫為準。

- All endpoint paths below are **relative to your API base URL**. The base URL may sit at a domain root or under a sub-path —

e.g. `http://localhost:8000/api`, `https://wsXX-YYYY-module-a.foredu.cn/api`.

Your API must work regardless of where it is mounted; you will enter your base URL in the test suite environment.

下列所有 Endpoint paths 皆為 API Base URL 的相對路徑。Base URL 可能位於網域根目錄或特定子路徑下 — 例如 `http://localhost:8000/api` 或 `https://wsXX-YYYY-module-a.foredu.cn/api`。無論 API 掛載於何處，系統都必須能正常運作；你需在測試套件的环境變數中輸入你的 Base URL。

- API documentation (Swagger UI) is available in `dist/api-docs/` — open `index.html` in your browser

API 文件 (Swagger UI) 位於 `dist/api-docs/` — 請用瀏覽器開啟 `index.html` 檢視。

- All responses must be JSON — no HTML responses anywhere
所有回應都必須是 JSON——任何地方都不得回傳 HTML
- All staff endpoints require Bearer token authentication (via `POST /login`)
所有專員端點都需要 Bearer token 驗證（透過 `POST /login`）
- All passenger portal endpoints require separate passenger Bearer token authentication (via `POST /passenger/login`)
所有旅客入口網站端點都需要獨立的旅客 Bearer token 驗證（透過 `POST /passenger/login`）
- Public endpoints `GET /claims/track/{referenceCode}` and `POST /passenger/register` require no authentication
公開端點 `GET /claims/track/{referenceCode}` 與 `POST /passenger/register` 不需驗證

Testing Your Work 結果測試

The same Bruno test suite used for assessment is provided in `dist/api-tests/` — use it throughout the module to check your progress.

評分所用的同一套 Bruno 測試套件提供於 `dist/api-tests/` 請在整個模組期間持續

使用它來檢查。

1. Open Bruno → **Open Collection** → select the api-tests folder. If Bruno asks which JavaScript sandbox to use, choose **Developer Mode** — the reset step needs it

開啟 Bruno → Open Collection → 選擇 api-tests 資料夾。若 Bruno 詢問要使用哪個 JavaScript 沙盒，請選擇 Developer Mode—重設步驟需要它

2. Select the **Local** environment and fill in:

選擇 Local 環境並填入：

- baseUrl — wherever your API runs,
e.g. <http://localhost:8000/api> or <http://localhost/<folder>/api>
baseUrl — 你的 API 執行的位置，例如 <http://localhost:8000/api> 或 <http://localhost/<folder>/api>
- dbName, dbUser, dbPass (and dbHost / dbPort if not local defaults) — your MySQL connection, so the suite can reset your data for you
dbName、dbUser、dbPass（若非本機預設值，另加 dbHost / dbPort）— 你的 MySQL 連線，讓套件能為你重設資料
- mysqlPath — only if the mysql command is not on your PATH, set the full path to the MySQL command-line client
mysqlPath — 僅在 mysql 指令不在你的 PATH 中時，設定 MySQL 命令列客戶端的完整路徑

3. Test names carry the marking aspect (A1.1: ... A9.4:) and quote its wording — a green test is the same check the assessors will run, in the same order as the marking scheme

測試名稱帶有評分面向（A1.1: … A9.4:）並引用其文字——一個綠色測試就是評分者將執行的同一項檢查，且順序與評分表相同

Testing one part at a time (recommended while you build — you do not need the whole API working to test the part you are on). Right-click a folder → **Run**, in this order:

一次測試一個部分（建議在開發過程中使用—你不需要整個 API 都能運作，就

能測試你正在處理的部分)。右鍵點選資料夾 →Run，依此順序：

1. A - reset — re-imports the provided dump directly into your MySQL database. It needs none of your code, so it works from the first minute of the module (some tests change data — always start here)

A - reset — 直接將提供的傾印檔重新匯入你的 MySQL 資料庫。它不需要任何你的程式碼，因此從模組的一開始就能運作（部分測試會變更資料—請務必每次都從這裡開始）

2. A1 - auth — logs in and stores the tokens every other folder uses

A1 - auth — 登入並儲存其他所有資料夾都會用到的 token

3. The folder you are working on, e.g. A3 – items

你正在處理的資料夾，例如 A3 - items

A8 - passenger-portal handles its own passenger login, but still needs steps 1 and 2 first (some of its tests use the staff token).

A8 - passenger-portal 會自行處理旅客登入，但仍需先執行步驟 1 與 2（部分測試會用到專員 token）

Running everything: run from the collection root — folders execute in order, the database is reset from the dump at the start and end automatically, and the run is repeatable.

執行全部：從 collection 根目錄執行—資料夾會依序執行，資料庫會在開始與結束時自動從傾印檔重設，且該次執行可重複進行。

Authentication 驗證

Two separate authentication systems exist:

系統存在兩套獨立的驗證機制：

Staff Auth 專員驗證

POST /login

Request:

```
{"email": "admin1@reclaim-pvg.cn", "password": "admin123"}
```

Response 200:

```
{
```

```
"data": {  
  "token": "kJH8s2Lq...60chars",  
  "user": { "id": 1, "name": "...", "email": "...", "role": "admin" }  
}  
}
```

Response 401: {"message": "Invalid credentials"} — also for inactive accounts.

回應 401 : {"message": "Invalid credentials"}—停用帳號亦回傳此訊息。

POST /logout — Invalidate token. No request body. Response 200: {"message": "Logged out successfully"}

使 token 失效。無請求主體。回應 200 : {"message": "Logged out successfully"}

All authenticated endpoints require header: Authorization: Bearer {token}

所有需驗證的端點都需要標頭：Authorization: Bearer {token}

Unauthenticated requests return 401: {"message": "Unauthenticated"}
未驗證的請求回傳 401 : {"message": "Unauthenticated"}

Passenger Auth 旅客驗證

POST /passenger/register and POST /passenger/login — see Passenger Portal section. Separate token system from staff; staff tokens are rejected on /passenger/* endpoints and vice versa. One deliberate exception: GET /terminals accepts either token (see Terminals).

POST /passenger/register 與 POST /passenger/login—見「旅客入口網站」章節。與專員採用不同的 token 系統；在 /passenger/* 端點上專員 token 會被拒絕，反之亦然。有一個刻意設計的例外：GET /terminals 接受任一種 token（見「航廈」）。

Roles 角色

- **Admin** — full access to all endpoints and all data

Admin（管理員）— 可完整存取所有端點與所有資料

- **Agent** — sees only data for their assigned terminals. Can register items and process claims for their terminals. Assigned to **multiple terminals** (many-to-many); all queries scoped to their assigned terminals.

Agent（專員）— 只能看到其所指派航廈的資料。可為其航廈登記物品並處理認領申請。可指派至多個航廈（多對多）；所有查詢皆限縮於其指派的航廈範圍內。

Forbidden actions return 403: {"message": "Forbidden"}

被禁止的操作回傳 403：{"message": "Forbidden"}

Seed Data 種子資料

Terminals 航廈

Code	Name
PVG-T1	Terminal 1
PVG-T2	Terminal 2
PVG-S1	Satellite Hall S1
PVG-S2	Satellite Hall S2

Staff 員工

Email	Password	Role	Terminals
admin1@reclaim-pvg.cn	admin123	Admin	All
admin2@reclaim-pvg.cn	admin123	Admin	All

Email	Password	Role	Terminals
pvg.cn			
agent1@reclaim-pvg.cn	agent123	Agent	Terminal 1 (PVG-T1), Satellite Hall S1 (PVG-S1)
agent2@reclaim-pvg.cn	agent123	Agent	Terminal 2 (PVG-T2), Satellite Hall S2 (PVG-S2)

Passengers 旅客

Sample 範例:

Email	Password	Name	Country
passenger1@email.com	passenger123	Li Wei	China
passenger2@email.com	passenger123	Maria Santos	Portugal
passenger31@email.com	passenger123	Chen Ming (deactivated account)	China

Categories 種類

electronics, documents, luggage, clothing, jewellery, keys, other

Endpoints — Terminals (3 endpoints)

端點 — 航廈（3 個端點）

GET /terminals

All terminals with counts. No pagination. Agents see only their assigned terminals.

所有航廈及其計數。無分頁。專員只能看到其指派的航廈

```
{
  "data": [
    {
      "id": 1,
      "name": "Terminal 1",
      "code": "PVG-T1",
      "description": "International departures and arrivals, gates A-D",
      "status": "open",
      "items_in_storage_count": 14,
      "open_claims_count": 9,
      "created_at": "2026-...",
      "updated_at": "2026-..."
    }
  ]
}
```

open_claims_count counts claims with status submitted or under-review.

open_claims_count 計算狀態為 submitted 或 under-review 的認領申請數量

GET /terminals also accepts a **passenger token**: passengers

receive **all** terminals (this feeds the terminal selector in the passenger portal's claim form). Staff behaviour is unchanged — agents still see only their assigned terminals.

GET /terminals 也接受旅客 token：旅客會收到所有航廈（此資料用於旅客入口網站認領表單中的航廈選擇器）。專員行為不變——專員仍只能看到其指派的航廈。

GET /terminals/{id}

Single terminal (same fields). Response 404 if not found. Agents get 403 for

terminals not assigned to them.

單一航廈（相同欄位）。找不到時回應 404。若航廈未指派給該專員，回傳 403

GET /terminals/{id}/items

Found items at a terminal (paginated). Query: ?status=in-storage

某航廈的拾獲物品（分頁）。查詢字串：?status=in-storage

Endpoints — Found Items (5 endpoints)

端點 — 拾獲物品（5 個端點）

GET /items

List found items (paginated, 15 per page, newest first). Agents see only items at their assigned terminals.

列出拾獲物品（分頁，每頁 15 筆，由新到舊排列）。專員只能看到其指派航廈的物品

Query params: ?status=in-

storage&category=electronics&terminal_id=1&search=iphone — search matches brand or description.

查詢參數：?status=in-storage&category=electronics&terminal_id=1&search=iphone — search 比對 brand 或 description

```
{
  "data": [
    {
      "id": 1,
      "reference_code": "FI-K3M9P2QX",
      "terminal": { "id": 1, "name": "Terminal 1", "code": "PVG-T1" },
      "category": "electronics",
      "brand": "Apple",
      "colour": "black",
```

```

    "description": "iPhone 15 Pro, cracked screen protector",

    "found_on": "2026-07-01",

    "found_location": "Gate D68",

    "storage_shelf": "T1-R3-S07",

    "status": "in-storage",

    "registered_by": { "id": 3, "name": "Agent One" },

    "created_at": "2026-...",

    "updated_at": "2026-..."

  }

],

"links": { "..."},

"meta": { "current_page": 1, "last_page": 9, "per_page": 15, "total": 123 }

}

```

brand, colour, and storage_shelf are nullable.

brand、colour 與 storage_shelf 可為 null

GET /items/{id}

Single item (same fields). Response 404 if not found. Agents get 403 if the item is not at their terminals.

單一物品（相同欄位）。找不到時回應 404。若物品不在專員的航廈，回傳 403

POST /items

Register a found item. Reference code auto-generated (FI-XXXXXXX). Status starts as registered. Agents can only register items at their assigned terminals (403 otherwise). An activity log is auto-created.

登記一件拾獲物品。參考編號自動產生（FI-XXXXXXX）。狀態起始為 registered。專員只能在其指派的航廈登記物品（否則 403）。系統會自動建立一筆活動紀錄

```
{  
  
  "terminal_id": 1,  
  
  "category": "electronics",  
  
  "brand": "Apple",  
  
  "colour": "black",  
  
  "description": "iPhone 15 Pro, cracked screen protector",  
  
  "found_on": "2026-07-01",  
  
  "found_location": "Gate D68"  
  
}
```

brand and colour optional; found_on must not be in the future.

Response 201 with the created item.

brand 與 colour 為選填；found_on 不得為未來日期。回應 201，包含所建立的物品

PUT /items/{id}

Update item details. All fields optional — send only what changes (partial updates are allowed):

更新物品細節。所有欄位皆為選填—只傳送有變動的欄位（允許部分更新）：

```
{ "brand": "Sony", "colour": "navy", "storage_shelf": "T1-R2-S04" }
```

Updatable

fields: category, brand, colour, description, found_location, storage_shelf. Status and reference code cannot be changed here. Response 200: updated item.

可更新欄位：category、brand、colour、description、found_location、storage_shelf。此處不可變更狀態與參考編號。回應 200：已更新的物品

PATCH /items/{id}/status

All fields are sent in the JSON request body — there are no query parameters on this endpoint.

所有欄位都放在 JSON 請求主體中傳送—此端點沒有查詢參數

Moving an item into storage (`storage_shelf` is required — either already set on the item, or sent together with the status):

將物品移入儲存（需要 `storage_shelf`—可為物品上已設定的值，或與狀態一併傳送）：

```
{ "status": "in-storage", "storage_shelf": "T1-R3-S07" }
```

Retiring an item after the retention period:

保管期滿後撤銷物品

```
{ "status": "disposed" }
```

Allowed via this endpoint:

透過此端點允許的轉換：

- `registered` → `in-storage` — requires `storage_shelf` (422 without one)
`registered` → `in-storage` — 需要 `storage_shelf`（缺少時 422）
- `in-storage` → `donated` | `disposed` — **only when found_on is more than 60 days ago**; 422 {"message": "Item is still within the retention period"} otherwise
`in-storage` → `donated` | `disposed` — 僅當 `found_on` 為超過 60 天之前；否則 422 {"message": "Item is still within the retention period"}

`matched` and `returned` cannot be set directly (422) — they are managed by the claim workflow. Invalid transitions return 422. Response 200: updated item.

Every status change writes an activity log.

`matched` 與 `returned` 不可直接設定（422）—它們由認領申請流程管理。無效的轉換回傳 422。回應 200：已更新的物品。每次狀態變更都會寫入一筆活動紀錄。

Endpoints — Claims (4 endpoints)

端點 — 認領申請（4 個端點）

GET /claims

List claims (paginated, newest first). Agents see only claims for their assigned

terminals.

列出認領申請（分頁，新到舊排列）。專員只能看到其指派航廈的申請

Query params: ?status=submitted&category=electronics&terminal_id=1

```
{
  "data": [
    {
      "id": 1,
      "reference_code": "CL-A7B2C9DE",
      "passenger": { "id": 1, "first_name": "Li", "last_name": "Wei", "email":
"passenger1@email.com" },
      "terminal": { "id": 1, "name": "Terminal 1", "code": "PVG-T1" },
      "category": "electronics",
      "brand": "Apple",
      "colour": "black",
      "description": "Black iPhone with red case, lock screen photo of a dog",
      "lost_on": "2026-07-01",
      "flight_number": "MU583",
      "status": "under-review",
      "matched_item": null,
      "created_at": "2026-...",
      "updated_at": "2026-...",
      "resolved_at": null
    }
  ],
  "links": { "..."},
}
```

```
"meta": { "current_page": 1, "last_page": 5, "per_page": 15, "total": 61 }
}
```

matched_item is null until a match is confirmed, then: { "id": 12,

"reference_code": "FI-...", "storage_shelf": "T1-R3-S07" }.

matched_item 在確認配對前為 null，之後為：{ "id": 12, "reference_code": "FI-...",
"storage_shelf": "T1-R3-S07" }

GET /claims/{id}

Single claim (same fields). Response 404 if not found. Agents get 403 if the claim is not for their terminals.

單一認領申請（相同欄位）。找不到時回應 404。若申請不屬於專員的航廈，回傳 403

PATCH /claims/{id}/status

Request: {"status": "under-review"}

Allowed via this endpoint only:

僅透過此端點允許的轉換:

- submitted → under-review — an agent picks up the claim
submitted → under-review — 專員接手該申請
- matched → under-review — releases the match: the linked item returns to in-storage and matched_item becomes null
matched → under-review — 解除配對：連結的物品回到 in-storage，且 matched_item 變為 null

All other statuses are managed by dedicated endpoints (422 here). Invalid transitions return 422. Response 200: updated claim. Activity logs auto-created.

所有其他狀態皆由專屬端點管理（在此為 422）。無效的轉換回傳 422。回應 200：已更新的申請。自動建立活動紀錄

GET /claims/track/{referenceCode} (PUBLIC — no auth)公開-無須驗證

```
{
  "data": {
```

```
"reference_code": "CL-A7B2C9DE",

"status": "under-review",

"category": "electronics",

"terminal": { "name": "Terminal 1", "code": "PVG-T1" },

"lost_on": "2026-07-01",

"created_at": "2026-...",

"resolved_at": null

}

}
```

No passenger personal data is exposed. Response 404 if the reference code is not found.

不會揭露任何旅客個人資料。若找不到參考編號，回應 404

Endpoints — Matching & Resolution (4 endpoints)

端點 — 配對與結案（4 個端點）

These four endpoints are one workflow. A typical claim travels like this:

這四個端點構成一套流程。一筆典型的認領申請流程如下：

1. A passenger files a claim → status submitted
旅客提交認領申請 → 狀態 submitted
2. An agent picks it up → PATCH /claims/{id}/status → under-review
專員接手 → PATCH /claims/{id}/status → under-review
3. The agent asks the system for likely items → GET
/claims/{id}/matches (ranked suggestions)
專員向系統詢問可能的物品 → GET /claims/{id}/matches（排序後的建議）
4. The agent confirms one → POST /claims/{id}/match → claim **and** item both become matched

專員確認其中一項 → POST /claims/{id}/match → 申請與物品皆變為 matched

5. Then one of three things happens:

接著會發生三種情況之一：

- the passenger collects it → POST /claims/{id}/resolve → claim resolved, item returned
旅客領取 → POST /claims/{id}/resolve → 申請 resolved，物品 returned
- it was the wrong item → PATCH /claims/{id}/status back to under-review → link cleared, item back to in-storage
配到錯誤物品 → PATCH /claims/{id}/status 退回 under-review → 連結清除，物品回到 in-storage
- nothing suitable exists → POST /claims/{id}/reject (open claims only)
沒有合適的物品 → POST /claims/{id}/reject（僅限未結案的申請）

GET /claims/{id}/matches

The matching engine. Given an open claim, it scans the stored items and returns the most likely matches, best first, so an agent can see at a glance which shelf to check.

配對引擎。給一筆未結案的申請，掃描已保管的物品並回傳最可能的配對，最佳者在前，讓專員一眼就能看出該檢查哪個架位

The claim must be submitted or under-review — 422 otherwise.

申請必須是 submitted 或 under-review—否則 422

Step 1 — candidates. Only items with status in-storage and the **same category** as the claim are considered. All other items are ignored.

步驟 1 — 候選項。只考慮狀態為 in-storage 且與申請同類別的物品。其餘物品一律忽略

Step 2 — score each candidate by comparing it with the claim (maximum 100 points):

步驟 2 — 為每個候選項評分，方法是將其與申請比較（最高 100 分）：

Comparison	Points
brand identical (case-insensitive; both values set)	30
colour identical (case-insensitive; both values set)	25
item is at the claim's terminal	20
days between found_on and lost_on: 0-1 days	25
days between found_on and lost_on: 2-3 days	15
days between found_on and lost_on: 4-7 days	5
days between found_on and lost_on: 8 days or more	0

Step 3 — result. Candidates scoring **less than 40 are dropped**. The rest are returned sorted by score (highest first); ties broken by found_on (newest first), then id (lowest first). Each entry carries the total score and its breakdown.

步驟 3 — 結果。分數低於 40 者剔除。其餘依 score 排序（最高者在前）；同分時以 found_on 決勝（最新在前），再以 id（最小在前）。每一筆項目都帶有總 score 及其 breakdown

In pseudocode — implement exactly this, in any language:

以下為虛擬碼—請以任何語言完全依此實作:

candidates = items where status = "in-storage" and category = claim.category

for each item in candidates:

breakdown.brand = 30 if claim.brand and item.brand are both set
and equal ignoring case, else 0

breakdown.colour = 25 if claim.colour and item.colour are both set
and equal ignoring case, else 0

breakdown.terminal = 20 if item.terminal_id equals claim.terminal_id, else 0

days = absolute difference in whole days between item.found_on and claim.lost_on

breakdown.date = 25 if days <= 1

15 if days <= 3

5 if days <= 7

0 otherwise

score = breakdown.brand + breakdown.colour + breakdown.terminal +
breakdown.date

keep only results with score >= 40

sort by score (high to low), then found_on (new to old), then id (low to high)

return each as { item, score, breakdown }

Worked example 實作範例 — claim: jewellery, Cartier, gold, Terminal 1, lost on 1 July:

Candidate (all jewellery, in storage)	brand	colour	terminal	date	Score
Cartier gold bracelet, Terminal 1, found 1 July	30	25	20	25	100
Cartier gold necklace, Terminal	30	25	0	15	70

Candidate (all jewellery, in storage)	brand	colour	terminal	date	Score
2, found 29 June					
Tiffany silver ring, Terminal 1, found 1 July	0	0	20	25	45
Pandora rose-gold bracelet, Terminal 2, found 21 June	0	0	0	0	0 — not returned

Response 200:

```
{
  "data": [
    {
      "item": {
        "id": 12,
        "reference_code": "FI-K3M9P2QX",
        "terminal": { "id": 1, "name": "Terminal 1", "code": "PVG-T1" },
        "category": "jewellery",
        "brand": "Cartier",
        "colour": "gold",
        "description": "...",
        "found_on": "2026-07-01",
        "storage_shelf": "T1-R3-S07"
```

```

    },
    "score": 100,
    "breakdown": { "brand": 30, "colour": 25, "terminal": 20, "date": 25 }
  }
]
}

```

An empty data array is returned when nothing scores 40 or more.

當沒有任何項目分數達 40 以上時，回傳空的 data 陣列

Check yourself: the worked example above is real — in the provided seed data, claim CL-F8DA73A7 (id 1) is exactly this jewellery claim. A correct implementation returns four suggestions scoring **100, 70, 50, 45** for it (the 50 is a brandless gold item not shown in the table). If your endpoint produces those numbers in that order, your engine is right.

自我檢查：上述實例演練為真實案例—在提供的種子資料中，申請 CL-F8DA73A7（id 1）正是這筆珠寶認領申請。正確的實作會為它回傳四筆建議，分數依序為 100、70、50、45（其中 50 分為表中未列出、無品牌的金色物品）。若你的端點依該順序產生這些數字，代表你的引擎正確

POST /claims/{id}/match

Request: {"item_id": 12}

Confirm a match. Claim must be under-review; item must be in-storage (422 otherwise — the item does not need to appear in the suggestions). Sets claim to matched, item to matched, links them. Activity logs auto-created for both.

確認配對。申請必須為 under-review；物品必須為 in-storage（否則 422—物品不需要出現在建議清單中）。將申請設為 matched、物品設為 matched，並將兩者連結。為兩者自動建立活動紀錄

Response 200: updated claim with matched_item.

回應 200：附帶 matched_item 的更新後申請

POST /claims/{id}/resolve

Hand the item back. No request body. Claim must be matched (422 otherwise). Sets claim to resolved (auto-sets resolved_at), item to returned. Activity logs auto-created.

將物品交還。無請求主體。申請必須為 matched（否則 422）。將申請設為 resolved（自動設定 resolved_at）、物品設為 returned。自動建立活動紀錄

Response 200: updated claim.

回應 200：更新後的申請

POST /claims/{id}/reject

Request: {"reason": "No matching item found after 30 days"} — reason optional.

Request: {"reason": "No matching item found after 30 days"} — reason 為選填

Claim must be submitted or under-review (422 otherwise; a matched claim must be released first). Sets claim to rejected. Activity log auto-created (reason in details).

申請必須為 submitted 或 under-review（否則 422；matched 狀態的申請必須先解除配對）。將申請設為 rejected。自動建立活動紀錄（原因記於 details 中）

Response 200: updated claim.

回應 200：更新後的申請

Endpoints — Dashboard (3 endpoints)

端點 — 儀表板（3 個端點）

GET /dashboard/stats

Aggregated statistics. Agents see only their assigned terminals' data.

彙總統計。專員只能看到其指派航廈的資料

```
{  
  
  "data": {  
  
    "total_items": 123,  
  
    "items_in_storage": 67,  
  
    "total_claims": 61,
```

```
"open_claims": 20,

"today_items": 7,

"today_claims": 2,

"items_by_category": {

  "electronics": 18,

  "documents": 23,

  "luggage": 20,

  "clothing": 21,

  "jewellery": 5,

  "keys": 14,

  "other": 22

},

"claims_by_status": {

  "submitted": 10,

  "under-review": 10,

  "matched": 7,

  "resolved": 20,

  "rejected": 8,

  "withdrawn": 6

},

"recent_claims": [

  {

    "id": 61,

    "reference_code": "CL-...",
```

```

        "passenger_name": "Li Wei",

        "category": "electronics",

        "status": "submitted",

        "date": "2026-..."

    }

]

}

}

```

open_claims = submitted + under-review. recent_claims contains the **5 most recent** claims, newest first.

open_claims = submitted + under-review 。 recent_claims 包含最近 5 筆申請，最新在前。

GET /dashboard/activity

20 most recent activity log entries, newest first. Agents see only logs for their terminals' items and claims. Staff actions only — every entry has an associated user. Passenger self-service actions (logged with no user) are not shown here.

最近 20 筆活動紀錄，最新在前。專員只能看到其航廈物品與申請的紀錄。僅限專員操作—每一筆紀錄都有關聯的 user。旅客的自助操作（記錄時無 user）不會顯示於此

```

{

  "data": [

    {

      "id": 1,

      "action": "claim_matched",

      "details": "Claim CL-A7B2C9DE matched to item FI-K3M9P2QX",

      "item": { "id": 12, "reference_code": "FI-K3M9P2QX" },

    }

  ]

}

```

```
"claim": { "id": 1, "reference_code": "CL-A7B2C9DE" },  
  
"user": { "id": 3, "name": "Agent One", "role": "agent" },  
  
"created_at": "2026-..."  
  
}  
  
]  
  
}
```

item or claim may be null depending on the action.

item 或 claim 可能為 null，視動作而定

GET /my-terminals (agents only)

All terminals assigned to the agent, with items registered today and open claims.

專員所指派的所有航廈，含今日登記的物品與未結案的申請。

```
{  
  
  "data": [  
  
    {  
  
      "terminal": {  
  
        "id": 1,  
  
        "name": "Terminal 1",  
  
        "code": "PVG-T1",  
  
        "description": "...",  
  
        "status": "open"  
  
      },  
  
      "todays_items": [  
  
        {  
  
          "id": 1,  
  
          "reference_code": "FI-...",  
  
        },  
  
      ],  
  
    },  
  
  ],  
  
}
```



```
        "category": "...",
        "description": "...",
        "status": "registered",
        "found_location": "..."
    }
],
"open_claims": [
    {
        "id": 1,
        "reference_code": "CL-...",
        "category": "...",
        "status": "submitted",
        "passenger_name": "Li Wei"
    }
]
}
```

Response 403 for admins.

管理員存取時回應 403

Endpoints — Passenger Portal (9 endpoints)

端點 — 旅客入口網站（9 個端點）

Separate authentication system for passengers. All endpoints use the /passenger/* prefix. Passengers can only access their own data.

旅客使用獨立的驗證系統。所有端點皆使用 `/passenger/*` 前綴。旅客只能存取自己的資料

POST /passenger/register (PUBLIC — no auth) 公開—無需驗證

```
{  
  
  "first_name": "Chen",  
  
  "last_name": "Jing",  
  
  "email": "chen.jing@email.com",  
  
  "phone": "+8613912345678",  
  
  "address_1": "88 Century Avenue",  
  
  "address_2": null,  
  
  "city": "Shanghai",  
  
  "postcode": "200120",  
  
  "country": "China",  
  
  "password": "mypassword1"  
}
```

phone and address_2 optional. Email must be unique (422); password minimum 8 characters. Response 201 — the passenger is logged in immediately:

phone 與 address_2 為選填。Email 必須唯一（否則 422）；密碼至少 8 個字元。回應 201—旅客會立即登入：

```
{  
  
  "data": {  
  
    "token": "...",  
  
    "passenger": {  
  
      "id": 31,  
  
      "first_name": "Chen",
```

```

    "last_name": "Jing",

    "email": "chen.jing@email.com",

    "phone": "+8613912345678",

    "address_1": "88 Century Avenue",

    "address_2": null,

    "city": "Shanghai",

    "postcode": "200120",

    "country": "China"

  }

}

}

```

POST /passenger/login

Request: { "email": "passenger1@email.com", "password": "passenger123" }

Response 200: same shape as register. Response 401 for invalid credentials or inactive accounts.

回應 200：形狀與 register 相同。憑證無效或帳號停用時回應 401

POST /passenger/logout

No request body. Response 200: {"message": "Logged out successfully"}

無請求主體。回應 200：{"message": "Logged out successfully"}

GET /passenger/claims

Own claims (paginated, newest first). Query: ?status=submitted. Same claim structure as GET /claims but **without** the passenger object.

自己的申請（分頁，最新在前）。查詢字串：?status=submitted。申請結構與 GET /claims 相同，但不含 passenger 物件

GET /passenger/claims/{id}

Single own claim. Response 404 if not found **or not owned** by the authenticated

passenger.

自己的單一申請。若找不到或非本人所有，回應 404

POST /passenger/claims

File a claim. Reference code auto-generated (CL-XXXXXXXX), status submitted.

An activity log is auto-created (with no user).

提交認領申請。參考編號自動產生（CL-XXXXXXXX），狀態為 submitted。自動建立活動紀錄（無 user）

```
{  
  
  "terminal_id": 1,  
  
  "category": "electronics",  
  
  "brand": "Apple",  
  
  "colour": "black",  
  
  "description": "Black iPhone with red case, lock screen photo of a dog",  
  
  "lost_on": "2026-07-01",  
  
  "flight_number": "MU583"  
  
}
```

brand, colour, flight_number optional; lost_on must not be in the future.

Response 201 with the created claim.

brand、colour、flight_number 為選填；lost_on 不得為未來日期。回應 201，附帶所建立的申請

POST /passenger/claims/{id}/withdraw

Own claim only. No request body. Claim must be submitted or under-review (422 otherwise). Sets status to withdrawn. Response 200: updated claim.

Activity log auto-created (no user).

僅限自己的申請。無請求主體。申請必須為 submitted 或 under-review（否則 422）。將狀態設為 withdrawn。回應 200：更新後的申請。自動建立活動紀錄（無 user）

GET /passenger/profile

```
{  
  "data": {  
    "id": 1,  
    "first_name": "Li",  
    "last_name": "Wei",  
    "email": "passenger1@email.com",  
    "phone": "+8613800138000",  
    "address_1": "200 Huaihai Middle Road",  
    "address_2": "Apt 12B",  
    "city": "Shanghai",  
    "postcode": "200021",  
    "country": "China"  
  }  
}
```

Never exposes password or api_token.

不可揭露 password 或 api_token

PUT /passenger/profile

All fields optional — send only what changes:

所有欄位皆為選填—只傳送有變動的欄位:

```
{ "phone": "+8613800138099", "city": "Shanghai" }
```

Updatable 可更新欄位 fields: first_name, last_name, email (unique 唯一), phone, address_1, address_2, city, postcode, country, password (min 至少 8 chars 字元).

Response 200: updated profile (same shape as GET /passenger/profile).

回應 200：更新後的個人資料（形狀與 GET /passenger/profile 相同）

Response Structure

Success 成功: {"data": ...} — single object or array 單一物件或陣列

Paginated 分頁: {"data": [...], "links": {...}, "meta": {"current_page": 1, "last_page": ..., "per_page": 15, "total": ...}} **Error** 錯誤:

Status	Meaning	Response
401	Unauthenticated	{"message": "Unauthenticated"}
403	Forbidden	{"message": "Forbidden"}
404	Not found	{"message": "Resource not found"}
422	Validation failed	{"message": "Validation failed", "errors": {...}}
422	Business rule violated	{"message": "<descriptive message>"}

The automated test suite (Bruno) asserts the quoted messages above — and every message quoted verbatim elsewhere in this document — character for character. Copy them exactly as written; a spelling difference fails the test.

Where the table says <descriptive message>, the wording is your own and only the status code is asserted.

自動化測試套件（Bruno）會逐字元比對上表中引用的訊息—以及本文件其他各處逐字引用的每一則訊息。請完全照原文複製；一個拼字差異就會導致測試失敗。表中標示 <descriptive message> 之處，文字由你自訂，僅比對狀態碼。

General Requirements 一般要求

- All responses must be JSON
所有回應都必須是 JSON

- **Error messages quoted verbatim in this document must match exactly** (for example Unauthenticated, Invalid credentials, Item is still within the retention period) — they are part of the specification and are asserted by the test suite. Where this document says "descriptive message" without quoting text, the wording is yours; only the status code is asserted

本文件中逐字引用的錯誤訊息必須完全一致（例如 Unauthenticated、Invalid credentials、Item is still within the retention period）——它們是規格的一部分，並由測試套件比對。凡本文件寫「descriptive message（描述性訊息）」而未引用具體文字之處，文字由你自訂；僅比對狀態碼

- **Responses must use the exact HTTP status codes specified for each endpoint** — successful GET requests return 200; creations (POST /items, POST /passenger/claims, POST /passenger/register) return 201; all other successful requests return 200. Framework defaults that differ (for example NestJS returning 201 on every POST) must be overridden to match

回應必須使用各端點所指定的確切 HTTP 狀態碼——成功的 GET 請求回傳 200；建立資源（POST /items、POST /passenger/claims、POST /passenger/register）回傳 201；所有其他成功的請求回傳 200。與此不同的框架預設值（例如 NestJS 對每個 POST 都回傳 201）必須覆寫以符合規格

- **Paginated endpoints: 15 items per page**
分頁端點：每頁 15 筆
- **Staff Bearer token authentication on all staff endpoints**
所有專員端點採用專員 Bearer token 驗證
- **Separate passenger Bearer token authentication on all passenger portal endpoints; the two token systems are not interchangeable**
所有旅客入口網站端點採用獨立的旅客 Bearer token 驗證；兩套 token 系統不可互通
- **Public endpoints (register, claim tracking) require no authentication**
公開端點（register、認領追蹤）無需驗證

- **Role-based access: admin sees all, agent scoped to assigned terminals**
角色型存取控制：管理員可看到全部，專員限縮於指派的航廈
- **Passenger endpoints scoped to the authenticated passenger's own data**
旅客端點限縮於已驗證旅客本人的資料
- **Activity logs auto-generated on item registration, item status changes, claim filing, claim status changes, match, resolve, reject, withdraw**
於物品登記、物品狀態變更、認領申請提交、申請狀態變更、配對、結案、駁回、撤回時自動產生活動紀錄
- **Item and claim status transitions enforced — invalid transitions return 422**
強制執行物品與申請的狀態轉換—無效的轉換回傳 422
- **Match suggestions computed exactly as specified — same scores, filtering, and ordering**
配對建議完全依規格計算—分數、篩選與排序皆相同
- **CORS headers enabled for cross-origin requests**
為跨來源請求啟用 CORS 標頭
- **Passwords stored hashed**
密碼以雜湊形式儲存
- **30 endpoints total**
總計 30 個端點